

IFT436 - Série d'exercices #5 : les algorithmes gloutons

Manuel Lafond

Exercice 1: Dans une journée, un certain professeur X a t plages de rendez-vous disponibles, chacune pouvant être allouée à un étudiant qui nécessite un entretien. Chaque étudiant a une plage maximum possible, e.g. un étudiant demande un rendez-vous avant la plage i , avec $1 \leq i \leq t$. Il s'avère que X est une personne ignoble, et affecte une importance différente à chaque étudiant. Le but est d'affecter chacune des t plages de temps à un étudiant de façon à maximiser la somme des importances des étudiants traités.

Formellement, on a le problème suivant:

Problème du professeur discriminant

Entrée: un ensemble $E = \{(x_1, p_1), (x_2, p_2), \dots, (x_n, p_n)\}$, où chaque (x_i, p_i) représente un étudiant: x_i est le temps maximum possible pour un rendez-vous, et p_i est l'importance de l'étudiant.

Sortie: un tableau $T = [(x'_1, p'_1), \dots, (x'_t, p'_t)]$ comprenant des éléments distincts de E , où $T[i]$ est l'étudiant assigné au temps i . On peut permettre $T[i] = \text{null}$ si on n'affecte personne à la plage i . On exige que pour tout $i \in \{1, \dots, t\}$, si l'élément $T[i] = (x'_i, p'_i)$ est non-null, il satisfait $i \leq x_i$. On veut que $\sum_{i=1}^k p'_i$ soit maximum parmi tous les choix possibles.

Considérez l'algorithme glouton suivant qui insère les étudiants en ordre d'importance, lorsque c'est possible, le plus tard possible.

fonction

```
professeurDiscriminant( $E = \{(x_1, p_1), (x_2, p_2), \dots, (x_n, p_n)\}, t$ )
  Trier  $E$  selon les importances  $p_i$ , en ordre décroissant
   $T = [null] * t$ 
  pour  $i = 1$  à  $n$  faire
    //trouver l'endroit le plus tard possible pour  $E[i] = (x_i, p_i)$ 
    //On commence par essayer  $x_i$ , puis  $x_i - 1$ , etc.
    pour  $j = x_i$  à 1 faire
      si  $T[j] = null$  alors
         $T[j] = (x_i, p_i)$ 
        break // Sortir de la boucle pour  $j$ 
    fin
  fin
  return  $T$ 
```

Montrez que cet algorithme est optimal, c'est-à-dire qu'il maximise la somme des importances.

Exercice 2: Supposons que l'on vit à une époque où les pièces de monnaie sont 25 sous, 10 sous, 5 sous et 1 sou. On doit rendre à un client un certain nombre de sous t en minimisant le nombre de pièces redonnées. Par exemple pour rendre 98 sous, on redonnerait $3 \times 25 + 2 \times 10 + 3 \times 1$ pièces. Plus généralement, on pourrait avoir un ensemble d'entiers P dénotant les valeurs de pièces disponibles (donc dans l'exemple, $P = \{25, 10, 5, 1\}$). L'algorithme glouton fait la chose suivante:

```
fonction minPieces( $t, P$ )  $t$  est la somme cible,  $P$  les valeurs de pièces
   $R = ()$ 
   $s = 0$ 
  tant que  $s < t$  faire
    Soit  $p$  la valeur maximum de  $P$  telle que  $s + p \leq t$ 
     $s = s + p$ 
    Ajouter  $p$  à  $R$ 
  fin
  return  $R$ 
```

Montrez que cet algorithme fonctionne correctement avec $P = \{25, 10, 5, 1\}$. Montrez ensuite qu'il ne fonctionne pas sur certaines entrée P .

Exercice 3: L'algorithme de Kruskal trouve un arbre couvrant minimum dans un graphe valué. Il n'est pas identique à l'algorithme de Prim, car il maintient une forêt (i.e. un ensemble d'arbres) au lieu d'un arbre partiel. À chaque étape, l'idée est d'ajouter l'arête de coût minimum qui ne crée pas de cycle dans la solution courante.

```

fonction algoDeKruskal( $G = (V, E, f)$ )
     $T = (V, \emptyset)$  // Notre sous-graphe partiel.
    tant que  $T$  a moins de  $n - 1$  arêtes faire
        Soit  $e$  l'arête de coût minimum telle que l'ajout de  $e$  à  $T$  ne
            créé pas de cycle
        Ajouter  $e$  à  $T$ 
    fin
    return  $T$ 

```

Montrez que l'algorithme de Kruskal trouve bel et bien un arbre couvrant minimum.